

SEMESTER END EXAMINATIONS - APRIL 2023

Program	: B.E :- Computer Science and Engineering	Semester	: III
Course Name	: Data Structures	Max. Marks	: 100
Course Code	: CS33	Duration	: 3 Hrs

Instructions to the Candidates:

- Answer one full question from each unit.

UNIT - I

- Write a C function to dynamically allocate two dimensional array. Mention how the pointer to pointer concept is used there. CO1 (07)
 - Explain sparse matrices and write function to transpose sparse matrix with an example. CO1 (07)
 - Analyze any five string related library functions with appropriate examples. CO1 (06)
- Identify the criterias to be satisfied by algorithms. CO1 (07)
 - Illustrate KMP pattern matching algorithm with an example. CO1 (07)
 - Write a C function to perform the fast transpose operation over a sparse matrix. CO1 (06)

UNIT - II

- Apply infix to postfix conversion algorithm to convert the following infix expressions to postfix expressions:
i. $(A+B)*C(D/(J+D))$ ii. $A+(B*C-(D/E^F)*G)*H$ CO2 (08)
 - Why do we need dynamic arrays for Stack implementation? Justify your answer with an example. CO2 (06)
 - Illustrate add and delete operations on a Circular queue with suitable examples. CO2 (06)
- Use the postfix evaluation algorithm to evaluate the following expression:
i. $5\ 6\ 2\ +\ *\ 12\ 4\ 1\ -$
ii. $2\ 3\ *\ 5\ 4\ *\ +\ 9\ -$
Tabulate the input, stack contents and output. CO2 (08)
 - Identify the limitations of a linear Queue with suitable examples. CO2 (06)
 - Write the C function to implement multiple stacks push() and pop() operations. CO2 (06)

UNIT - III

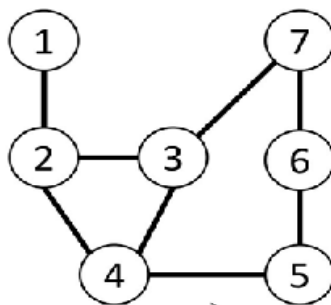
- Write a graphical representation to show the basic operations of a List. CO3 (06)
 - Describe the advantages of linked representation of data over sequential representation with suitable examples. CO3 (06)
 - Write a C functions to implement Stacks using a singly linked list with header nodes. CO3 (08)
- Show the sparse matrix representation using linked list. CO3 (06)
 - Illustrate a doubly linked circular list with a header node using a neat diagram. CO3 (06)
 - Write C functions to implement multiple Queues represented as a linked list. CO3 (08)

UNIT- IV

- | | | | | |
|----|----|---|-----|------|
| 7. | a) | Define binary tree. Write any three properties of binary trees. | CO4 | (06) |
| | b) | Differentiate between max heap and min heap with an example. | CO4 | (06) |
| | c) | Write an iterative C function to search for an element in a binary search tree. | CO4 | (08) |
| 8. | a) | Illustrate with examples how to transform a forest into a binary tree. | CO4 | (06) |
| | b) | Explain the memory representation of a threaded binary tree with an example. | CO4 | (06) |
| | c) | Construct a binary search tree stepwise for the following list of numbers: 24, 12, 6, 18, 30, 54, 7, 10, 42, 36. Perform inorder, preorder and postorder traversal on the constructed binary search tree. | CO4 | (08) |

UNIT - V

- | | | | | |
|----|----|---|-----|------|
| 9. | a) | Inspect the two ways of representing the following graph. | CO5 | (07) |
|----|----|---|-----|------|



- | | | | | |
|-----|----|--|-----|------|
| | b) | List out the operations over the following and describe:
i) Single ended priority queue
ii) Double ended priority queue. | CO5 | (07) |
| | c) | Describe about weight biased leftist tree with appropriate example. | CO5 | (06) |
| 10. | a) | Write a c function to perform depth first search over a graph. Show the dfs() function using an example. | CO5 | (07) |
| | b) | Illustrate height biased leftist tree with an example. | CO5 | (07) |
| | c) | Inspect the balance factor usage in AVL tree with an example. | CO5 | (06) |
